Edit Distance - Levenshtein Distance

The minimum number of single-character edits (insert, delete, substitute) needed to transform string A into string B.

Example: convert „kitten" to „sitting"

Let A = „kitten" (length $m=6$), B = „sitting" (length $n=7$).

Define $dp[i][j]$ = edit distance between $A[0..i-1]$ and $B[0..j-1]$.

Base cases:
$dp[0][j] = j$ (insert j characters into empty string)
$dp[i][0] = i$ (delete i characters to get empty string)

Recurrence:

If $A[i-1] = B[j-1]$: $dp[i][j] = dp[i-1][j-1]$ (characters match, free!)

Otherwise: $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$

The three operations:
- $dp[i-1][j] + 1$: delete $A[i-1]$
- $dp[i][j-1] + 1$: insert $B[j-1]$
- $dp[i-1][j-1] + 1$: substitute $A[i-1]$ to $B[j-1]$

		s	i	t	t	i	n	g
		s	i	t	t	i	n	g
k	K	1	2	3	4	5	6	7
i	i	2	1	2	3	4	5	6
t	t	3	2	1	2	3	4	5
t	t	4	3	2	1	2	3	4
e	e	5	4	3	2	2	3	4
n	n	6	5	4	3	3	3	3

The edit distance is dp[6][7] = 3.

Edit sequence: kitten → sitten (sub K→s) → sittin (sub e→i) → sitting (ins g)

Time complexity: O(m * n), Space: O(m * n) or O(min(m,n)) with optimization.

$\boxed{\text{Edit distance = minimum operations to transform one string to another. O}}$